
大数据与人工智能
自动化学院

Chapter03.05 nn的网络层介绍(卷积层，池化层，激活函数，全连接层等)

3.5-1 导言

- 上一篇的模型创建Module、模型容器Containers及AlexNet网络搭建， 我们已经知道了:
  - 如何搭建模型
  - 学习了关于搭建模型非常重要的一个类nn.Module和模型容器Containers
  - 搭建模型我们提到两个步骤:
    - 建立子模块
    - 拼接子模块

**本节我们再细一点，具体学习几个重要的子模块：比如卷积层，池化层，激活函数，全连接层等。

- 首先我们从卷积层开始， 学习一下1/2/3维度卷积
  - 然后学习一下nn.Conv2d这个方法
  - 最后介绍一下什么是转置卷积。
- 然后我们进行池化层的学习
  - 包括平均池化和最大池化
- 然后介绍一下全连接层，这个一般称为线性层，
- 最后我们介绍一些常用的非线性激活函数
  - 为什么会用到非线性激活函数?
  - 一句话，没有非线性激活函数，再深的神经网络也没有“深度”的意义。

大纲如下：

- 卷积运算与卷积层(1/2/3d卷积， nn.Conv2d, nn.ConvTranspose)
- 池化运算与池化层(最大池化和平均池化)
- 全连接层
- 非线性激活函数层
- 总结梳理

3.5-2. 卷积运算与卷积层

卷积层，得先从卷积运算开始

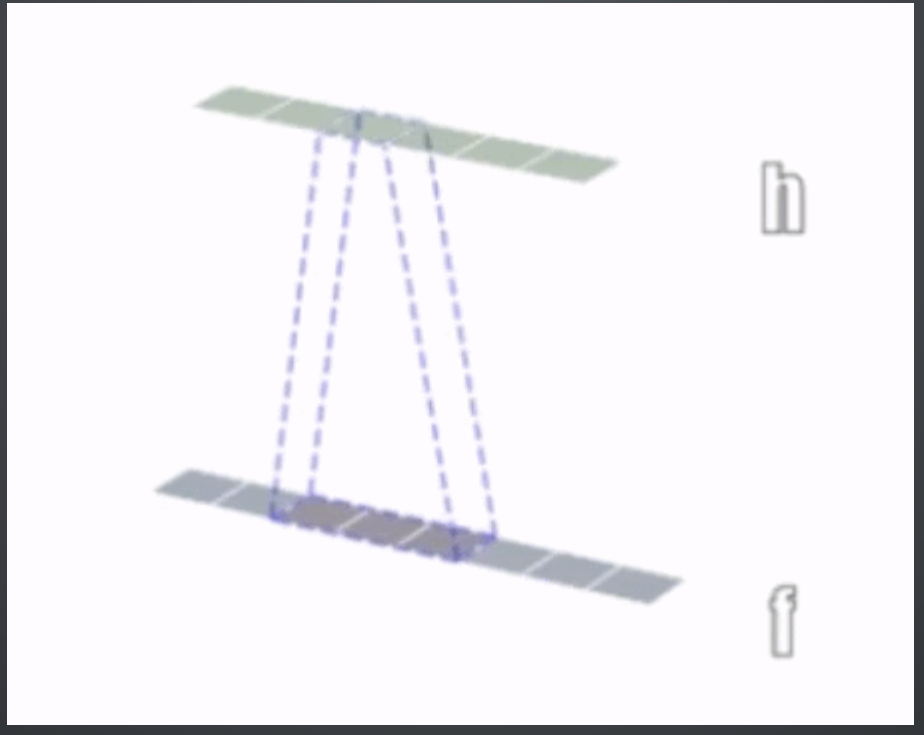
- 卷积运算就是卷积核在输入信号（图像）上滑动， 相应位置上进行**乘加**。
- 卷积核又称为滤过器，过滤器， 可认为是某种模式，某种特征。

下面我们就看看1维/2维/3维的卷积示意图

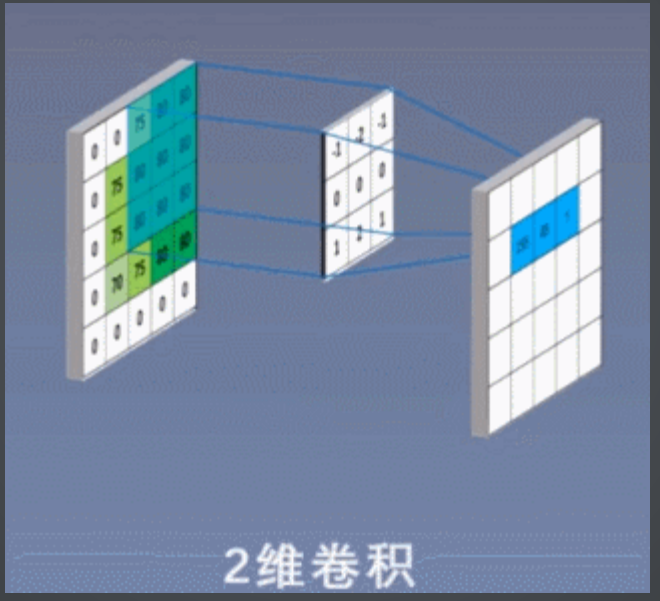
3.5-2.1 1维/2维/3维卷积示意

一般情况下，卷积核在几个维度上滑动，就是几维卷积。注意下面都是一个卷积核：

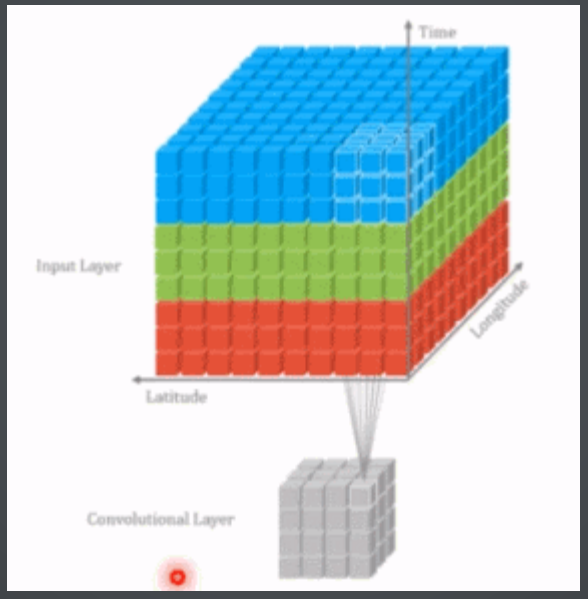
1. 一维卷积示意



2. 二维卷积示意



3. 三维卷积示意



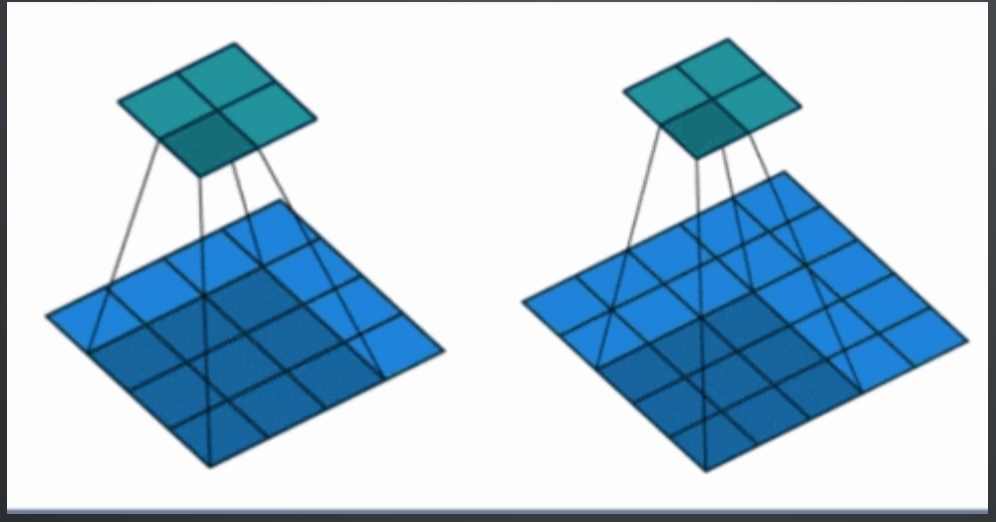
3.5-2.2 二维卷积函数nn.Conv2d

nn.Conv2d: 对多个二维信号进行二维卷积

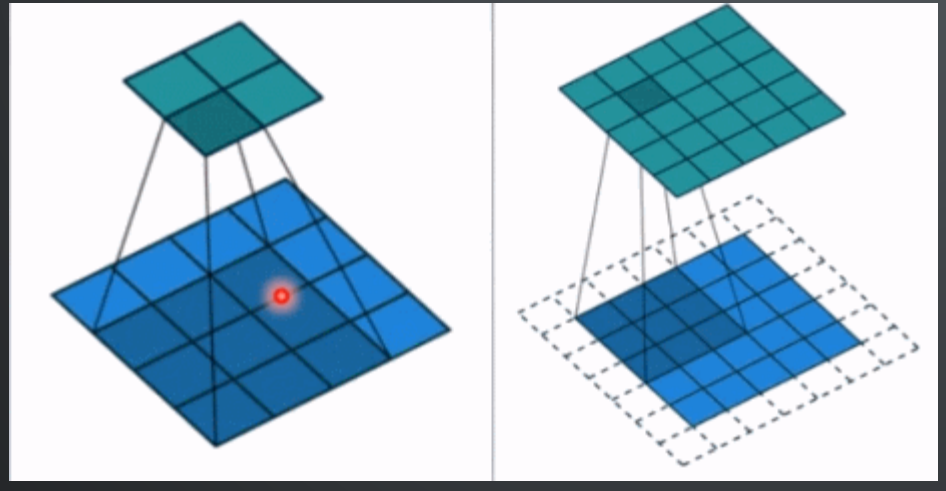
```
nn.Conv2d(in_channels,
          out_channels,
          kernel_size,
          stride=1,
          padding=0,
          dilation=1,
          groups=1,
          bias=True,
          padding_mode='zeros')
```

主要参数:

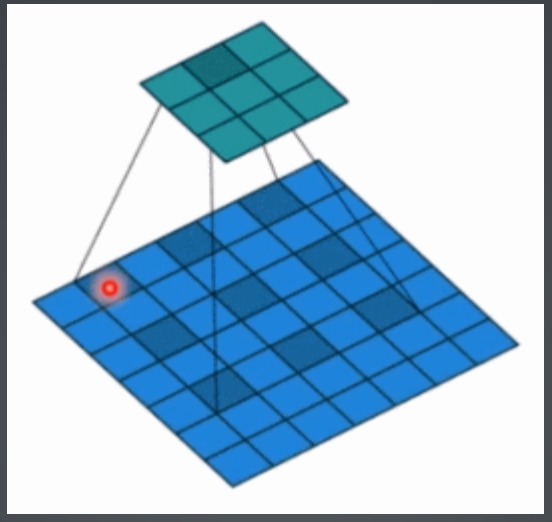
- in_channels: 输入通道数
- out_channels: 输出通道数， 等价于卷积核个数
- kernel_size: 卷积核尺寸， 这个代表着卷积核的大小
- stride: 步长， 这个指的卷积核滑动的时候，每一次滑动几个像素。 下面看个动图来理解步长的概念:
  - 左边那个的步长是1， 每一次滑动1个像素，
  - 而右边的步长是2，会发现每一次滑动2个像素。



- padding: 填充个数， 通常用来保持输入和输出图像的大小的匹配，
- 看左边那个图，这个是没有padding的卷积，输入图像是4 * 4， 经过卷积之后，输出图像就变成了2 * 2的了， 这样分辨率会遍变低，并且我们会发现这种情况卷积的时候边缘部分的像素参与计算的机会比较少。
- 加入考虑padding的填充方式，就是在原输入周围加入像素，这样就可以保证输出的图像尺寸分辨率和输入的一样，并且边缘部分的像素也受到同等的关注了。

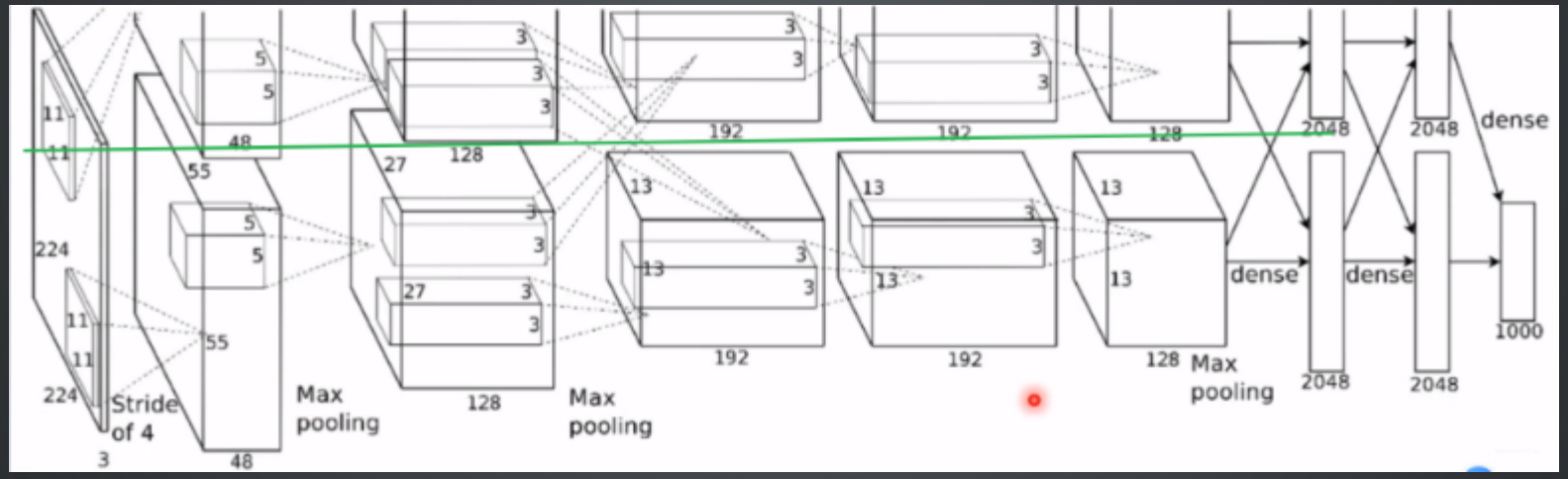


- dilation: 孔洞卷积大小， 下面依然是一个动图：



- 孔洞卷积就可以理解成一个带孔的卷积核， 常用于图像分割任务，主要功能就是提高感受的视野。
- 也就是输出图像的一个参数，能看到被处理的图像更大的一个区域。

- groups: 分组卷积设置， 分组卷积常用于模型的轻量化， 可以减少模型参数。
  - 比如下图的两组卷积分别进行提取，最后合并。



`bias`: 偏置。

- bias是torch.nn.Conv2d()的一个参数，它默认为True，
- 表示在卷积层的输出信号上加上一个偏置项（bias term），从而增加模型的非线性和表达能力。
- 偏置项是一个一维的张量（tensor），张量的shape等于卷积层的输出通道数（out_channels）

- 下面是卷积之后的尺寸计算的方式(一条边的大小)：

1. 没有padding：

$$\mathrm{out}_{\mathrm{size}} = \frac{\mathrm{In}_{\mathrm{size}} - \mathrm{kernel}_{\mathrm{size}}}{\mathrm{stride}} + 1$$

2. 如果有padding的话：

$$\mathrm{out}_{\mathrm{size}} = \frac{\mathrm{In}_{\mathrm{size}} + 2 \times \mathrm{padding}_{\mathrm{size}} - \mathrm{kernel}_{\mathrm{size}}}{\mathrm{stride}} + 1$$

3. 如果再加上孔洞卷积的话：

$$\mathrm{out}_{\mathrm{size}} = \frac{\mathrm{In}_{\mathrm{size}} + 2 \times \mathrm{padding}_{\mathrm{size}} - \mathrm{dilation}_{\mathrm{size}} \times (\mathrm{kernel}_{\mathrm{size}} - 1) - 1}{\mathrm{stride}} + 1$$

提问：关于二维卷积层nn.Conv2d的参数描述，以下哪个选项是正确的？

A. in_channels表示卷积核个数
B. out_channels表示输入通道数
C. kernel_size表示卷积核的尺寸
D. stride表示填充像素的数量

解答

正确答案：C

解答分析：

- in_channels表示输入通道数，例如RGB图像的输入通道数为3，而不是卷积核个数（错误）。
- out_channels表示输出通道数，即卷积核的个数（错误）。
- kernel_size表示卷积核的尺寸，例如3×3或5×5（正确）。
- stride表示卷积核滑动的步长，而不是填充像素数量（错误）.

下面我们用代码看看卷积核是怎么提取特征的，毕竟有图才有真相：

01_PyTorch_Practice > lesson3 > bigAI-03-05-20241124.py > ...

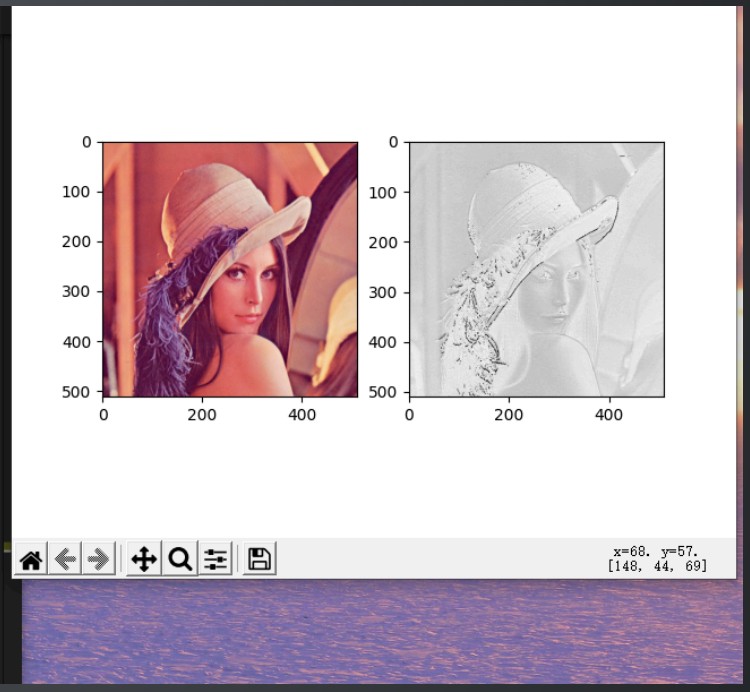
```python
# ================================ load img ==================================
path_img = os.path.join(os.path.dirname(os.path.abspath(__file__)), "imgs", "lena.png")
print(path_img)
img = Image.open(path_img).convert('RGB')  # 0~255

# convert to tensor
img_transform = transforms.Compose([transforms.ToTensor()])
img_tensor = img_transform(img)
# 添加 batch 维度
img_tensor.unsqueeze_(dim=0)    # C*H*W to B*C*H*W

# ================================ create convolution layer ==================================

# ================ 2d
# flag = 1
flag = 1
if flag:
    conv_layer = nn.Conv2d(3, 1, 3)   # input:(i, o, size) weights:(o, i , h, w)
    # 初始化卷积层权值
    nn.init.xavier_normal_(conv_layer.weight.data)
    # nn.init.xavier_uniform_(conv_layer.weight.data)

    # calculation
    img_conv = conv_layer(img_tensor)
```



接下来，我们改变seed， 也就是相当于换一组卷积核， 看看提取到什么样的特征：

01_PyTorch_Practice > lesson3 > bigAI-03-05-20241124.py > tt

```python
from matplotlib import pyplot as plt
from common_tools import transform_invert, set_seed

set_seed(5)  # 设置随机种子

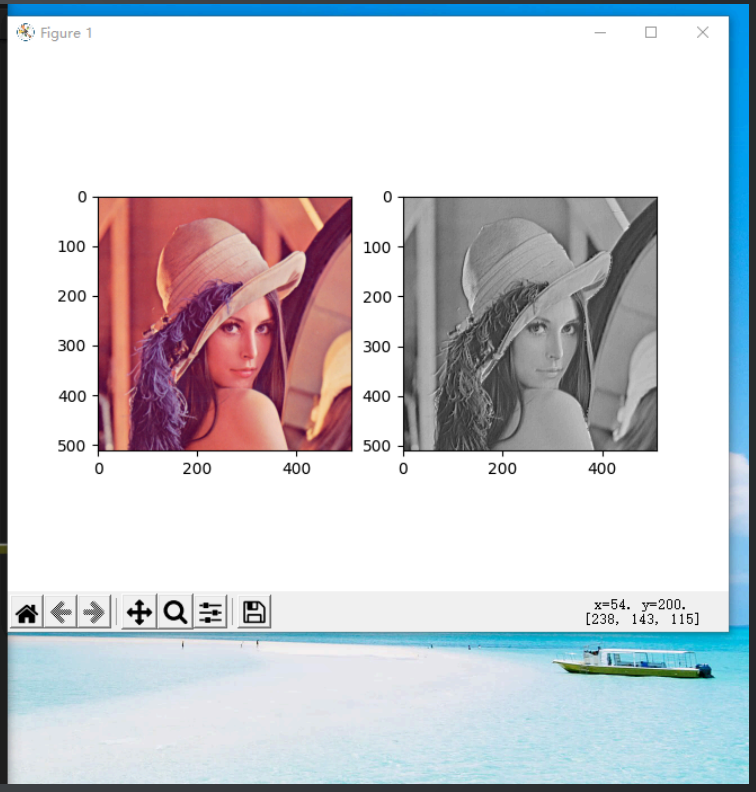
# ================================ load img ==================================
path_img = os.path.join(os.path.dirname(os.path.abspath(__file__)), "imgs", "lena.png")
print(path_img)
img = Image.open(path_img).convert('RGB')  # 0~255

# convert to tensor
img_transform = transforms.Compose([transforms.ToTensor()])
img_tensor = img_transform(img)
# 添加 batch 维度
img_tensor.unsqueeze_(dim=0)    # C*H*W to B*C*H*W

# ================================ create convolution layer ==================================

# ================ 2d
# flag = 1
flag = 1
if flag:
    conv_layer = nn.Conv2d(3, 1, 3)   # input:(i, o, size) weights:(o, i , h, w)
    # 初始化卷积层权值
    nn.init.xavier_normal_(conv_layer.weight.data)
    # nn.init.xavier_uniform_(conv_layer.weight.data)

    # calculation
    img_conv = conv_layer(img_tensor)
```



再换一个随机种子：

nn_layers_convolution.py

```
from tools.common_tools import transform_invert, set_seed

set_seed(1)  # 设置随机种子

# ================================ load img ==================================
path_img = os.path.join(os.path.dirname(os.path.abspath(__file__)), "lena.png")
img = Image.open(path_img).convert('RGB')  # 0~255

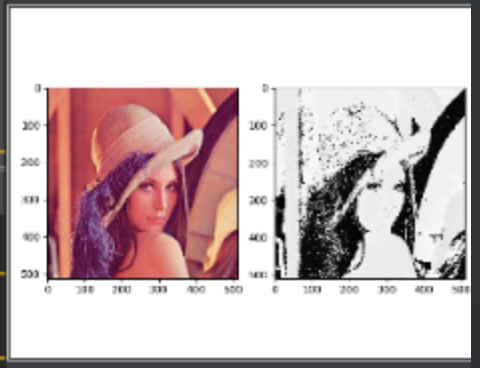
# convert to tensor
img_transform = transforms.Compose([transforms.ToTensor()])
img_tensor = img_transform(img)
img_tensor.unsqueeze_(dim=0)    # C*H*W to B*C*H*W

# ============================ create convolution layer ======================

# ============== 2d
flag = 1
# flag = 0
if flag:
    conv_layer = nn.Conv2d(3, 1, 3)   # input:(i, o, size) weights:(o, i , h, w)
    nn.init.xavier_normal_(conv_layer.weight.data)

    # calculation
    img_conv = conv_layer(img_tensor)
```

SciView: Data Plots



- **卷积层定义**：
  - `nn.Conv2d(3, 1, 3)` 表示：
    - 输入通道数为 3（RGB 图像）。
    - 输出通道数为 1（单通道特征图）。
    - 卷积核大小为 3×3
- **权重初始化**：
  - 使用 `nn.init.xavier_normal_` 初始化卷积层权重。
    - Xavier 初始化是一种常用的方法，目的是让权重分布均匀，避免梯度消失或爆炸。
    - 权重的值是基于随机数生成器产生的，因此受随机种子的影响。

通过上面，我们会发现不同权重的卷积核代表不同的模式，会关注不同的特征，这样我们只要设置多个卷积核同时对图片的特征进行提取，就可以提取不同的特征。

下面我们看一下图像尺寸的变化：

```
卷积前尺寸:torch.Size([1, 3, 512, 512])
卷积后尺寸:torch.Size([1, 1, 510, 510])
```

- 卷积前，图像尺寸是 512×512，
- 卷积后， 图像尺寸是 510×510。
- 我们这里的卷积核设置， 输入通道3， 卷积核个数1， 卷积核大小3， 无padding，步长是1， 那么我们根据上面的公式， 输出尺寸: $(512 - 3)/1 + 1 = 510$

- 我们知道卷积层也是继承于nn.Module的，所以肯定又是那几个字典属性
- 我们主要看看它的_modules参数和_parameters参数字典。

```python
if flag:
    conv_layer = nn.Conv2d(3, 1, 3)  # input:(i, o, size) weights:(o, i , h, w)
    # 初始化卷积层权值
    nn.init.xavier_normal_(conv_layer.weight.data)
    # nn.init.xavier_uniform_(conv_layer.weight.data)

    # calculation
    img_conv = conv_layer(img_tensor)

# ================ transposed
# flag = 1
flag = 0
if flag:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS VARIABLES

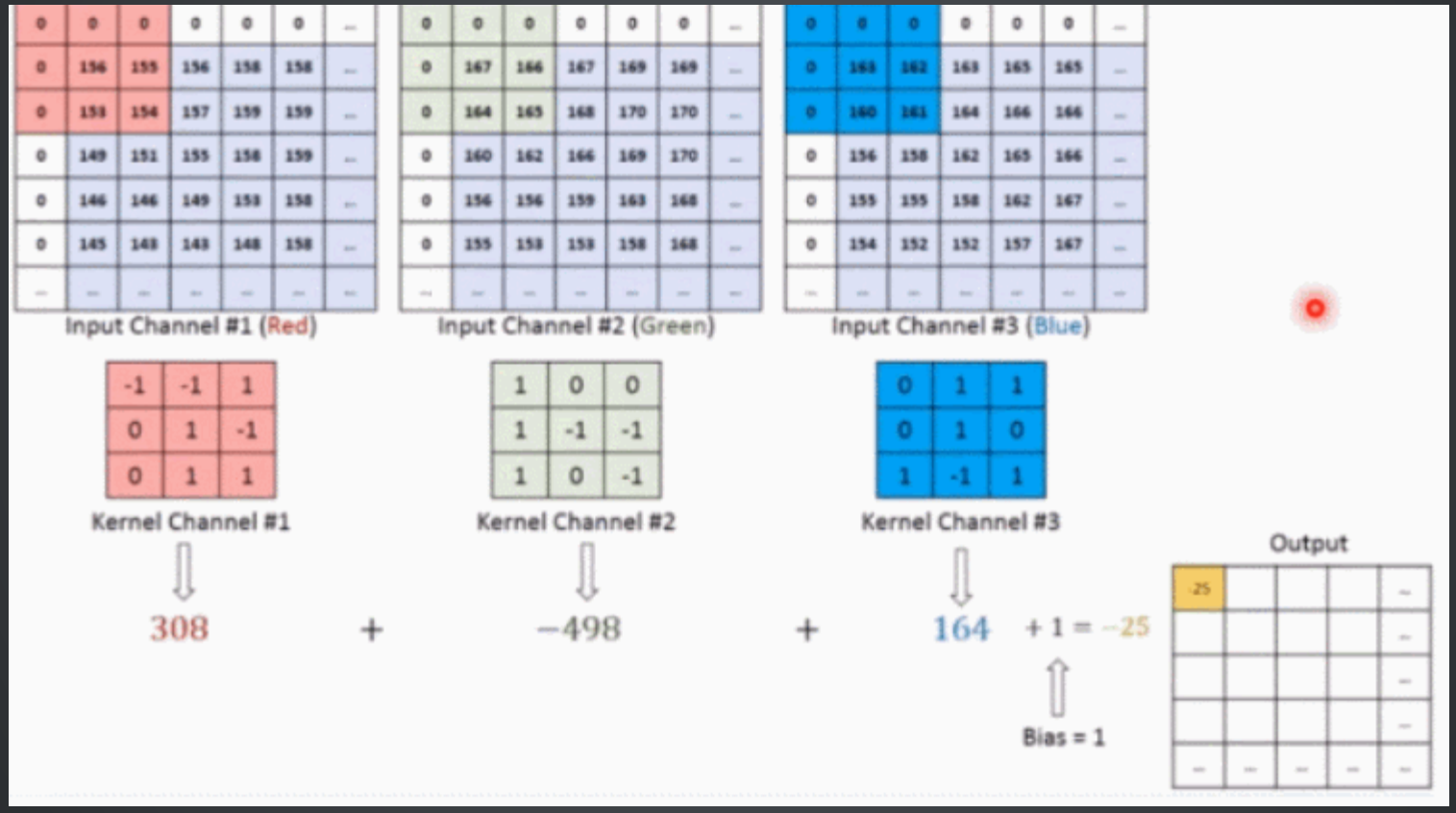
Locals

```
conv_layer = Conv2d(3, 1, kernel_size=(3, 3), stride=(1, 1))
  _parameters = OrderedDict([('weight', Parameter containing:
    'weight' = Parameter containing:
    > special variables
    > function variables
      H = 'Traceback (most recent call last):\n  File "c:\\Users\\strange007\\.vscode\\extensions\\ms-python.debugpy-20…
    > T = tensor([[[[ 0.1812],
    > data = tensor([[[[ 0.1812,  0.2176, -0.1316],
    > device = device(type='cpu')
    > dtype = torch.float32
```

我们可以看到Conv2d下面的_parameters存放着权重参数

- 这里的weight的形状是[1, 3, 3, 3]， 这个应该怎么理解呢?
- 首先1代表着卷积核的个数，
- 第1个3表示的输入通道数，
- 后面两个3是二维卷积核的尺寸。
- 那么这里有人可能会有疑问，我们这里是3维的卷积核啊，怎么实现的二维卷积呢?

下面再通过一个示意图看看：



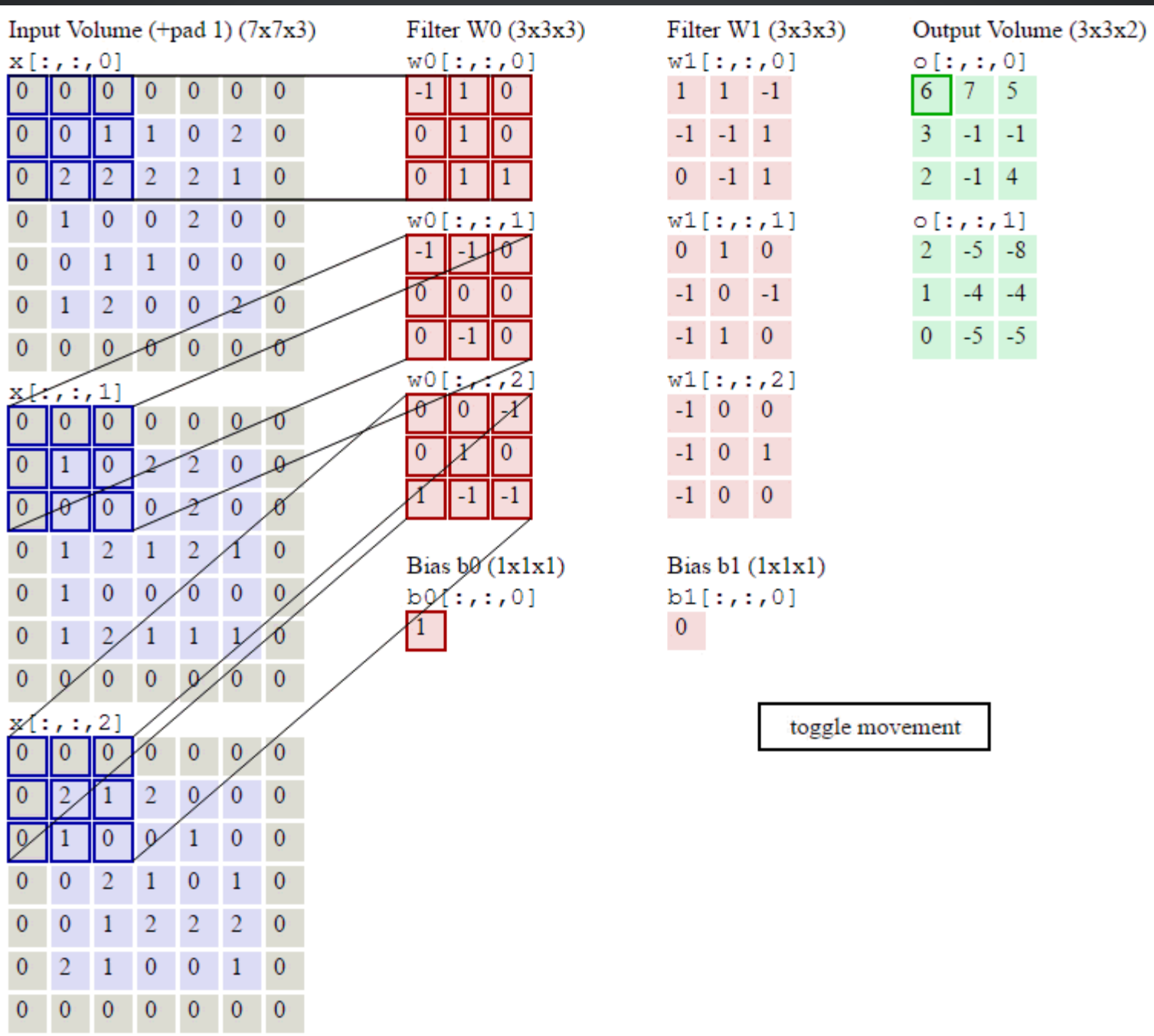
- 我们的图像是RGB3个通道的图像，所以有3个二维的卷积核，
- 这3个二维的卷积核分别对应一个通道进行卷积
    - 比如红色通道上，只有一个卷积核在上面滑动，
    - 每一次滑动，对应元素相乘然后相加得到一个数，
    - 这三个卷积核滑动一次就会得到三个数
    - 这三个数之和加上偏置才是我们的一个输出结果

这里我们看到了，一个卷积核只在2个维度上滑动，所以最后得到的就是2维卷积。

这也能理解开始的卷积维度的概念了（一般情况下，卷积核在几个维度上滑动，就是几维卷积）

下面看一下多个卷积核的提取特征(图片来自RGB彩色图像的卷积过程（gif动图演示)， 下面每一块扫描都是对应元素相乘再相加得到最后的的结果：

上面这一个是一个三维的卷积示意，并且使用了2个卷积核。最后会得到2个二维的张量。


Input Volume (+pad 1) (7x7x3)
x[:,:,0]
0 0 0 0 0 0 0
0 0 1 1 0 2 0
0 2 2 2 2 1 0
0 1 0 0 2 0 0
0 0 1 1 0 0 0
0 1 2 0 0 2 0
0 0 0 0 0 0 0
x[:,:,1]
0 0 0 0 0 0 0
0 1 0 2 2 0 0
0 0 0 0 2 0 0
0 1 2 1 2 1 0
0 1 0 0 0 0 0
0 1 2 1 1 1 0
0 0 0 0 0 0 0
x[:,:,2]
0 0 0 0 0 0 0
0 2 1 2 0 0 0
0 1 0 0 1 0 0
0 0 2 1 0 1 0
0 0 1 2 2 2 0
0 2 1 0 0 1 0
0 0 0 0 0 0 0
Filter W0 (3x3x3)
w0[:,:,0]
-1 1 0
0 1 0
0 1 1
w0[:,:,1]
-1 -1 0
0 0 0
0 -1 0
w0[:,:,2]
0 0 -1
0 1 0
1 -1 -1
Bias b0 (1x1x1)
b0[:,:,0]
1
Filter W1 (3x3x3)
w1[:,:,0]
1 1 -1
-1 -1 1
0 -1 1
w1[:,:,1]
0 1 0
-1 0 -1
-1 1 0
w1[:,:,2]
-1 0 0
-1 0 1
-1 0 0
Bias b1 (1x1x1)
b1[:,:,0]
0
Output Volume (3x3x2)
o[:,:,0]
6 7 5
3 -1 -1
2 -1 4
o[:,:,1]
2 -5 -8
1 -4 -4
0 -5 -5
toggle movement

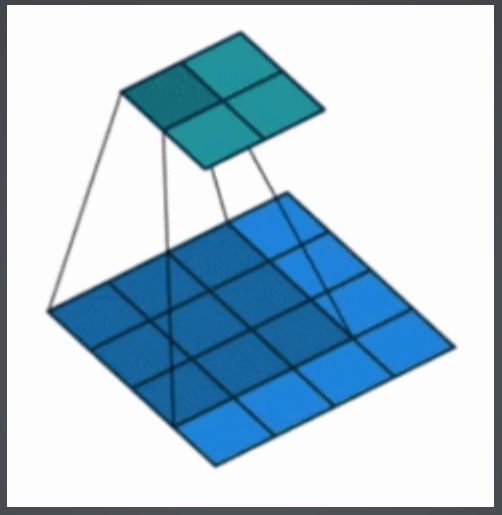
3.5-2.3 转置卷积

转置卷积又称为反卷积和部分跨越卷积。用于对图像进行上采样。在图像分割任务中经常被使用。

为什么它叫转置卷积呢?

- 先来看看正常的卷积在代码实现过程中的一个具体操作： 对于正常的卷积，我们需要实现大量的相乘相加操作，而这种乘加的方式恰好是矩阵乘法所擅长的。
- 所以在代码实现的时候，通常会借助矩阵乘法快速的实现卷积操作， 那么这是怎么做的呢?

我们假设图像尺寸为 4×4 , 卷积核为 3×3, padding=0, stride=1， 也就是下面这个图：



- 首先将图像尺寸的 4×4 拉长成 16×1 ， 16代表所有的像素，1代表只有1张图片。
- 然后 3×3 的卷积核会变成一个 4×16 的一个矩阵（变换方法下面讲）

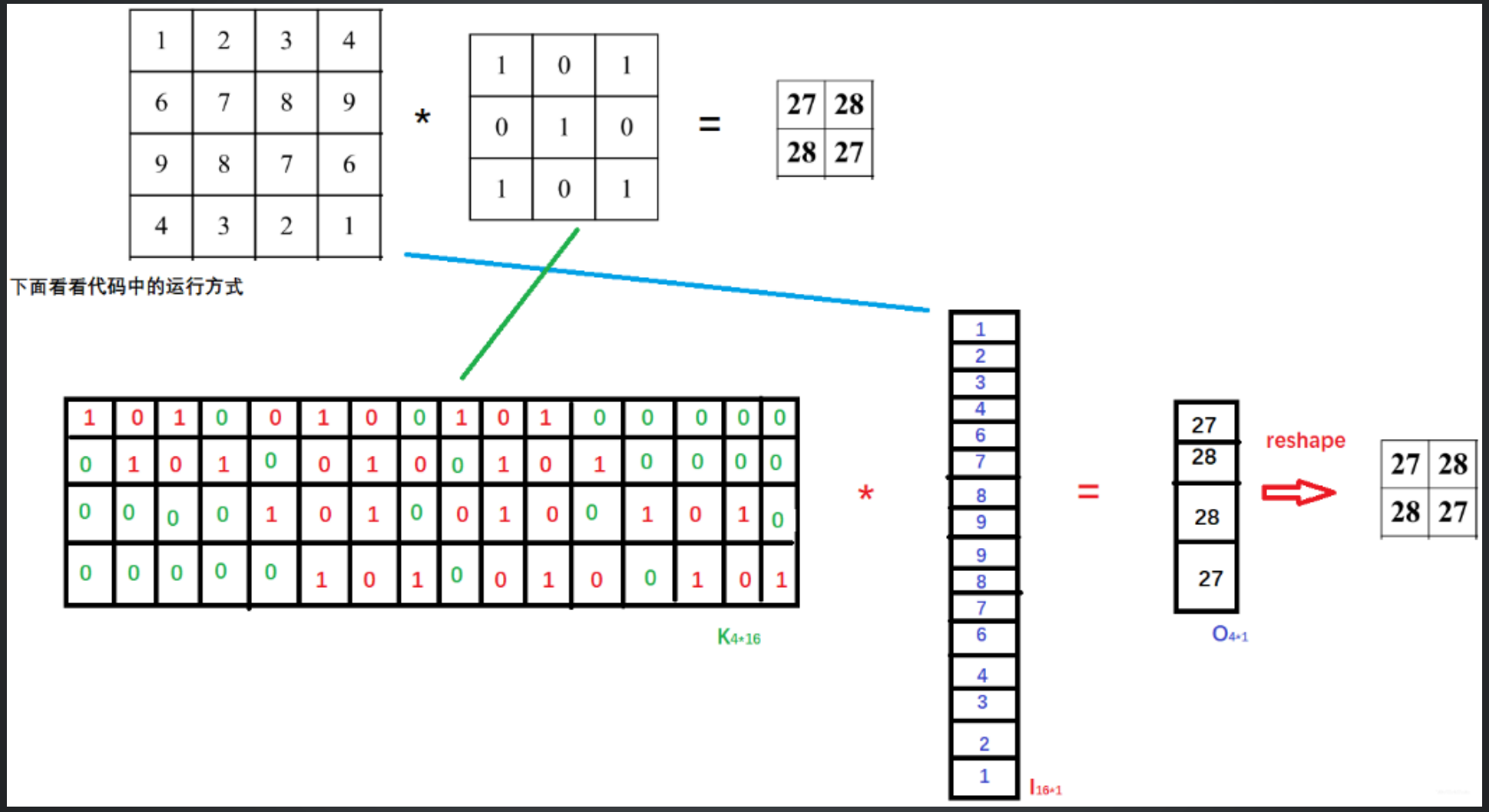
矩阵相乘实现卷积的过程：3 × 3 --> 4 × 16 这是怎么变的

- 首先这个16，是先把9个权值拉成一列
- 然后按位置补7个0变成16，
- 这个4是根据我们输出的尺寸计算的
  - 根据输入尺寸，卷积核大小，padding， stride信息可以得到输出单边的尺寸是 (4 – 3) / 1 + 1 = 2，
  - 所以输出是 2 × 2
  - 那么拉成一列就是4。
- 这样我们的输出：

$$O_{4\times1} = K_{4\times16} \times I_{16\times1}$$

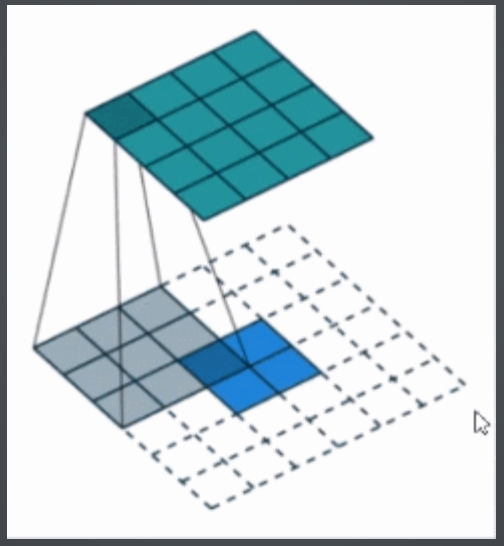
这样就得到了最后一列输出4个元素，然后reshape就得到了 2 × 2 的一个输出特征图了。

这就是用矩阵乘法输出一个二维卷积的这样一个示例。这就是那个过程：



下面我们看看转置卷积的上采样是怎么样的：

转置卷积是一个上采样：输入的图像尺寸是比较小的，经过转置卷积之后，会输出一个更大的图像。看下面示意图：



我们这里的输入图像尺寸是 2×2， 卷积核为 3×3，padding=0， stride=1, 我们的输出图像尺寸是 4×4

我们看看这个在代码中是怎么通过矩阵乘法进行实现的

- 首先，依然是把输入的尺寸进行拉长，成一个 4×1 的一个向量，
- 然后我们的卷积核会变成 16×4 的，注意这里的卷积核尺寸，这个4依然是根据卷积核得来的，记得上面的那个16吗？ 我们是把卷积核拉长然后补0， 在这里我们不是补0了，而是采用剔除的方法，因为我们根据上面的图像可以发现，虽然这里的卷积核有9个权值，可是能与图像相乘的最多只有四个（也就是卷积核在中间的时候）， 所以这里采用剔除的方法，从9个权值里面剔除5个，得到4个权重， 而这个16，依然是根据输出图像的尺寸计算得来的。
- 因为我们这里的输出是 4×4 ， 这个可以用上面尺寸运算的公式。

所以这里的输出：

$$O_{16\times1} = K_{16\times4} \times I_{4\times1}$$

- 这次注意这个卷积核的尺寸是 16×4，而我们正常卷积运算的卷积核尺寸 4×16，所以在形状上这两个卷积操作卷积核恰恰是转置的关系，这也就是转置卷积的由来了。
- 这是因为卷积核在转变成矩阵的时候，与正常卷积的卷积核形状上是互为转置，注意是形状，具体数值肯定是不一样的。

下面就具体学习Pytorch提供的转置卷积的方法：

nn.ConvTranspose2d: 转置卷积实现上采样

```
nn.ConvTranspose2d(in_channels,
                   out_channels,
                   kernel_size,
                   stride=1,
                   padding=0,
                   output_padding=0,
                   groups=1,
                   bias=True,
                   dilation=1,
                   padding_mode='zeros')
```

- in_channels: int，表示输入信号的通道数，也就是输入特征图的深度。
- out_channels: int，表示卷积产生的通道数，也就是输出特征图的深度。
- kernel_size: int or tuple，表示卷积核的大小
- stride: int or tuple, optional，表示卷积跨越的步长，也就是要将输入扩大。

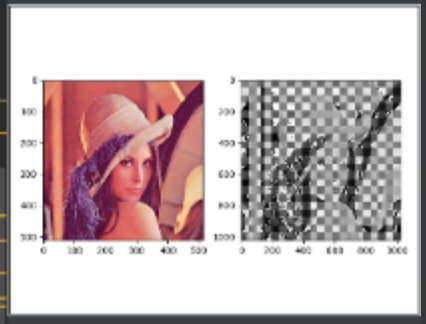
其他参数和卷积运算的参数差不多，就不再解释一遍了。

下面从代码中看看转置卷积怎么用：

nn_layers_convolution.py

```
37
38  # ================ transposed
39  flag = 1
40  # flag = 0
41  if flag:
42      conv_layer = nn.ConvTranspose2d(3, 1, 3, stride=2)   # input:(i, o, size)
43      nn.init.xavier_normal_(conv_layer.weight.data)
44
45      # calculation
46      img_conv = conv_layer(img_tensor)
47
48
49  # ================================ visualization ==================================
50  print("卷积前尺寸:{}\n卷积后尺寸:{}".format(img_tensor.shape, img_conv.shape))
51  img_conv = transform_invert(img_conv[0, 0:1, ...], img_transform)
52  img_raw = transform_invert(img_tensor.squeeze(), img_transform)
53  plt.subplot(122).imshow(img_conv, cmap='gray')
54  plt.subplot(121).imshow(img_raw)
55  plt.show()
```

SciView: Data Plots



- 转置卷积有个通病叫做“棋盘效应”，看上面图，
- 这是由于不均匀重叠导致的。
- 至于如何解决，搜索资料

关于尺寸变化：

```
卷积前尺寸:torch.Size([1, 3, 512, 512])
卷积后尺寸:torch.Size([1, 1, 1025, 1025])
```

我们发现，输入图像是512的， 卷积核大小是3，
==stride=2，所以输出尺寸：（512 – 1）× 2== + 3 = 1025

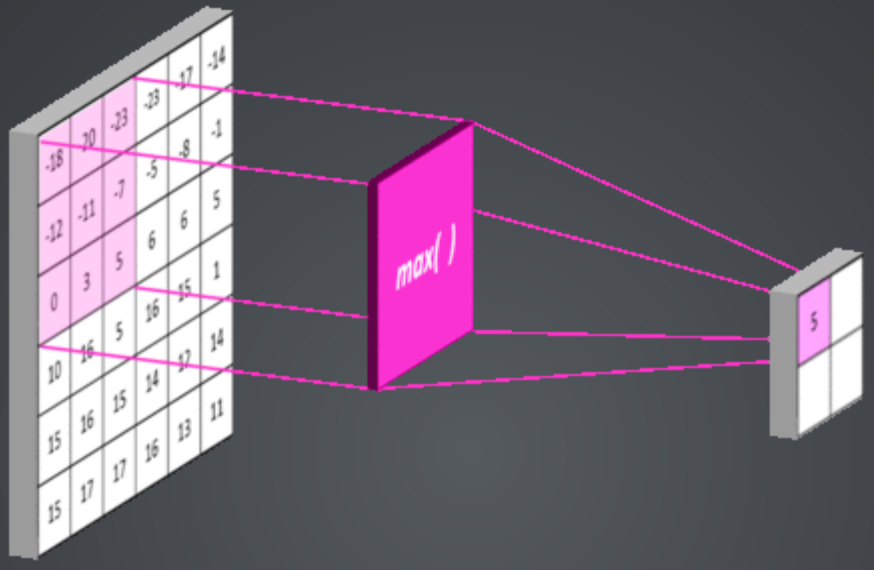
简单梳理，卷积部分主要是卷积运算， 卷积尺寸的计算，然后又学习了转置卷积。下面我们看看nn中其他常用的层。

3.5-3 池化层

池化运算：对信号进行“**收集**”并“**总结**”， 类似水池收集水资源， 因而美其名曰池化层。

- 收集： 多变少，图像的尺寸由大变小
- 总结： 最大值/平均值

下面是一个最大池化的动态图看一下



- 最大池化就是这些元素里面去最大的值作为最终的结果。
- 平均池化就是这些元素取平均值作为最终值

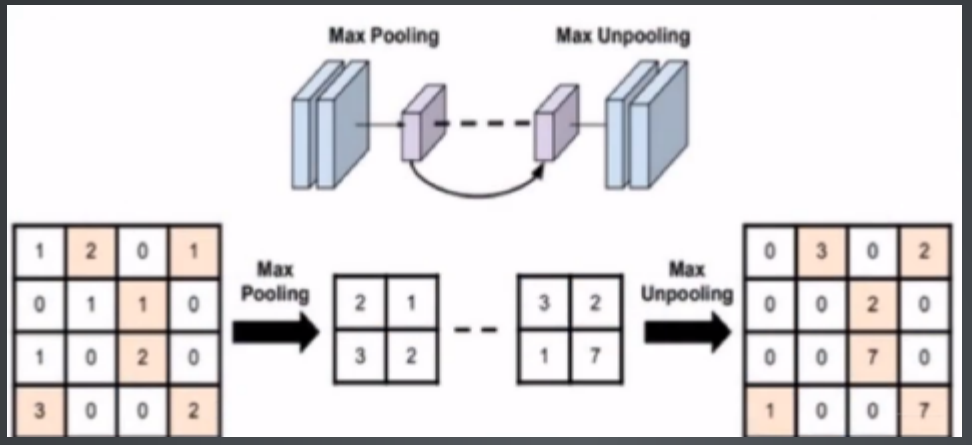
下面看看Pytorch提供的最大池化和平均池化的函数：
nn.MaxPool2d: 对二维信号(图像）进行最大值池化。

```
nn.MaxPool2d(kernel_size, stride=None,
             padding=0, dilation=1,
             return_indices=False,
             ceil_mode=False)
```

- kernel_size: 池化核尺寸
- stride: 步长
- padding: 填充个数
- dilation: 池化核间隔大小
- ceil_mode: 尺寸向上取整
- return_indices: 记录池化像素索引

前四个参数和卷积的其实类似， 最后一个参数常在最大值反池化的时候使用

那什么叫最大值反池化呢？看下图：



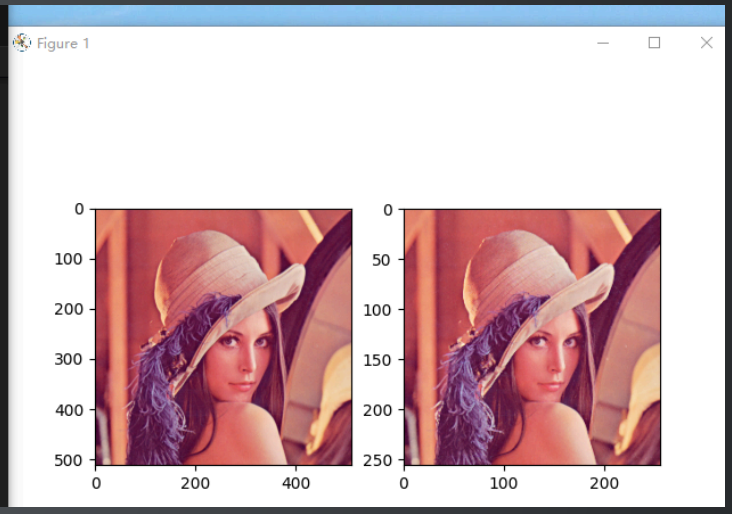
- 反池化就是将尺寸较小的图片通过上采样得到尺寸较大的图片
- 看右边那个， 那么这些元素放到什么位置呢？
  - 之前有做记录，记录最大值池化时候元素的位置
  - 然后在最大值反池化的时候把元素放回去那些位置

下面看一下最大池化的效果：

bigAI-03-05-20241124.py × launch.json

01_PyTorch_Practice > lesson3 > bigAI-03-05-20241124.py > tt

```
57
58  ######################
59  # 池化层
60  ######################
61
62  set_seed(1)  # 设置随机种子
63
64  # ================================ create convolution layer ================================
65
66  # ================ maxpool
67  flag = 1
68  # flag = 0
69  if flag:
70      maxpool_layer = nn.MaxPool2d((2, 2), stride=(2, 2))   # input:(i, o, size) weights:(o, i , h, w)
71      img_pool = maxpool_layer(img_tensor)
72
```



- 可以发现，图像基本上看不出什么差别，
- 但是图像的尺寸减少了一半，
- 所以池化层是可以帮助我们剔除一些冗余像素的。

除了最大池化，还有一个叫做平均池化：
nn.AvgPool2d: 对二维信号(图像)进行平均值池化

```
nn.AvgPool2d(kernel_size,
             stride=None,
             padding=0,
             ceil_mode=False,
             count_include_pad=True,
             divisor_override=None)
```

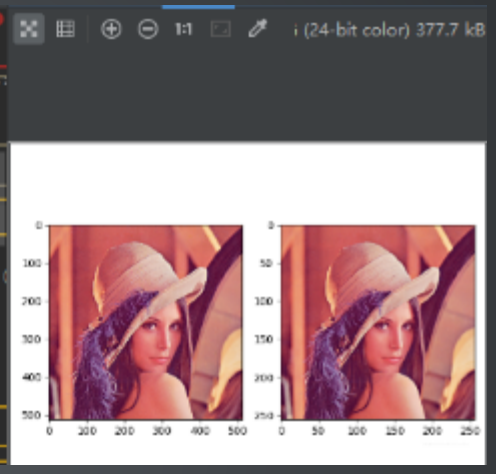
- count_include_pad: 当为True时，在计算平均值时会将填充的0也包含在内。默认值是True。
- divisor_override: 除法因子， 这个是求平均的时候那个分母，默认是有几个数相加就除以几，当然也可以自己通过这个参数设定

下面也是通过代码看一下结果：

```
flag = 0
if flag:
    maxpool_layer = nn.MaxPool2d((2, 2), stride=(2, 2))   # input:(i, o, size) weights:
    img_pool = maxpool_layer(img_tensor)

# ================ avgpool
flag = 1
#flag = 0
if flag:
    avgpoollayer = nn.AvgPool2d((2, 2), stride=(2, 2))  # input:(i, o, size) weights:(
    img_pool = avgpoollayer(img_tensor)

# ================ avgpool divisor_override
# flag = 1
flag = 0
if flag:
```



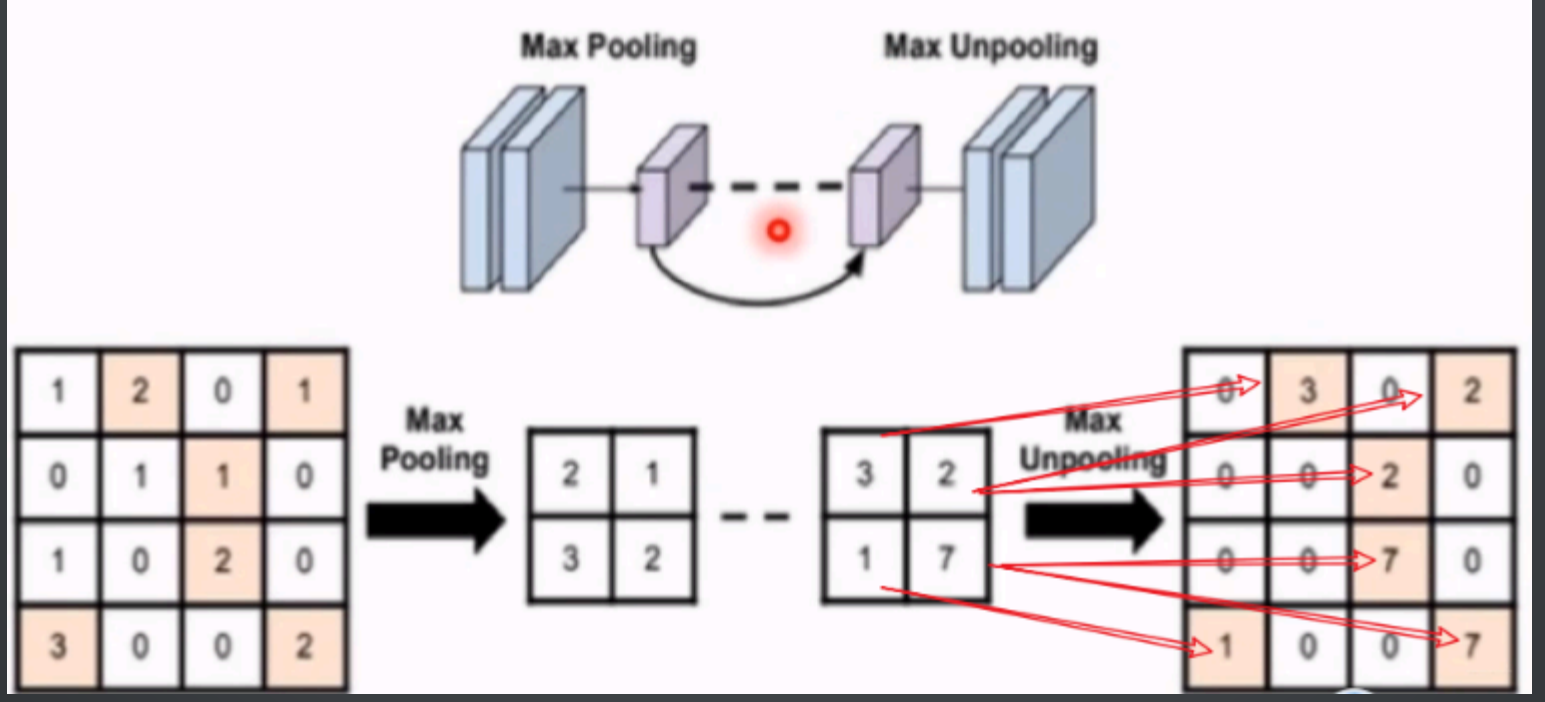
- 这个平均池化和最大池化在这上面好像看不出区别来，
- 其实最大池化的亮度会稍微亮一些，毕竟它都是取的最大值
- 而平均池化是取平均值。

- 这个的功能是对二维信号(图像)进行最大池化上采样

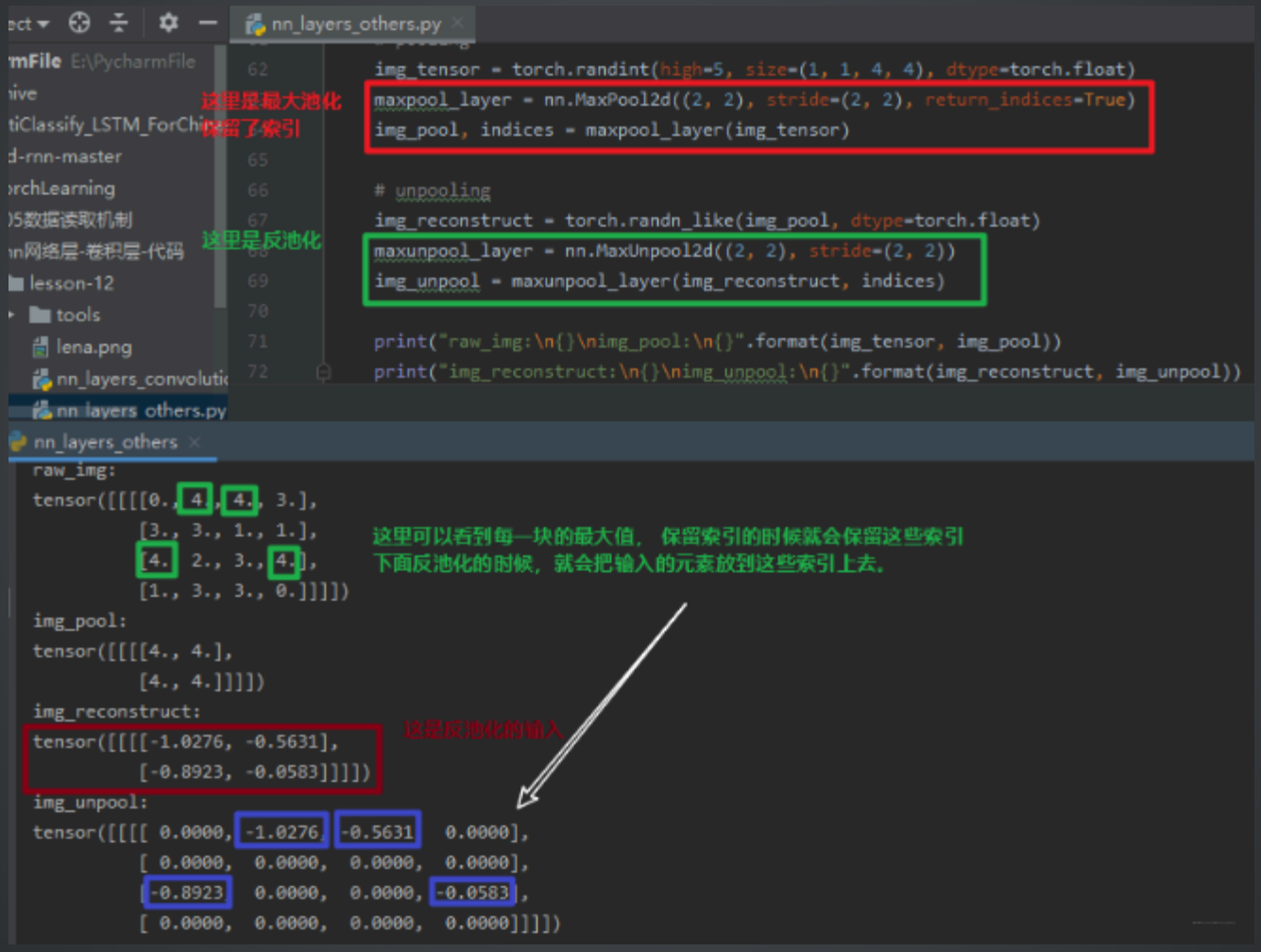
```
nn.MaxUnpool2d(kernel_size,
               stride=None,
               padding=0)

forward(self, input, indices, output_size=None)
```

- 这里的参数与池化层是类似的。
- 唯一的不同就是前向传播的时候我们需要传进一个indices， 我们的索引值，要不然不知道把输入的元素放在输出的哪个位置上
- 就像上面的那张图片：


Max Pooling
Max Unpooling
1 2 0 1
0 1 1 0
1 0 2 0
3 0 0 2
Max Pooling
2 1
3 2
3 2
1 7
Max Unpooling
0 3 0 2
0 0 2 0
0 0 7 0
1 0 0 7

下面通过代码来看一下反池化操作：



提问：以下关于池化层的描述，哪个是正确的？

A. 最大池化取的是池化区域内的平均值
B. 平均池化会导致梯度消失问题
C. 池化层可以减少图像尺寸并去除冗余信息
D. 最大值反池化用于对图像进行下采样

解答

正确答案：C

*解答分析：

- 最大池化取的是池化区域内的最大值，而不是平均值（错误）。
- 平均池化不会直接导致梯度消失问题，它只是取区域内像素值的平均值（错误）。
- 池化层通过对特征进行总结和收集，可以减少图像尺寸并去除冗余信息（正确）。
- 最大值反池化用于上采样，而非下采样（错误）

3.5-4 线性层

线性层又称为全连接层

- 其每个神经元与上一层所有神经元相连实现对前一层的**线性组合，线性变换**

线性变换的计算在前面讲线性回归有说过，这里直接学习Pytorch的线性模块。

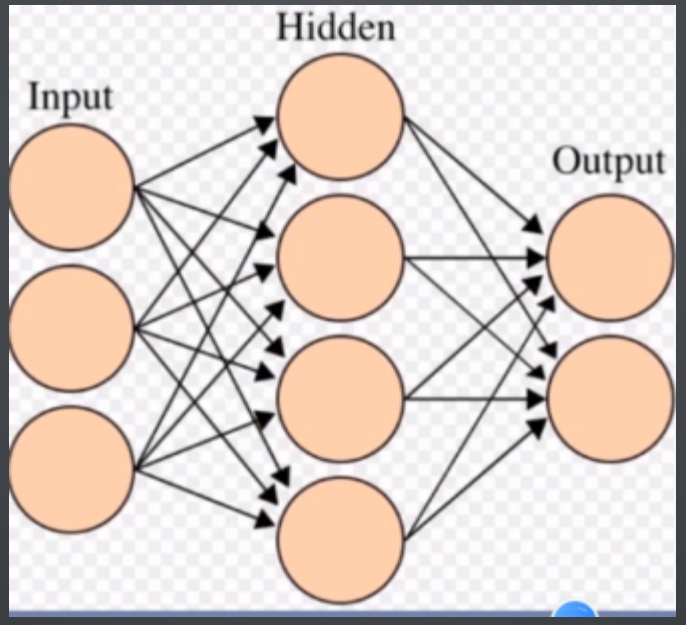
`nn.Linear(in_features, out_features, bias=True)`:
对一维信号（向量）进行线性组合

- in_features: 输入节点数
- out_features: 输出节点数
- bias: 是否需要偏置

计算公式：

$$y = xW^T + bias$$

(注意乘的是W的转置)



```
inputs = torch.tensor(1., 2, 3)
linear_layer = nn.Linear(3, 4)
linear_layer.weight.data = torch.tensor([[1.

linear_layer.bias.data.fill_(0.5)
output = linear_layer(inputs)
print(inputs, inputs.shape)
print(linear_layer.weight.data, linear_layer
```

结果:

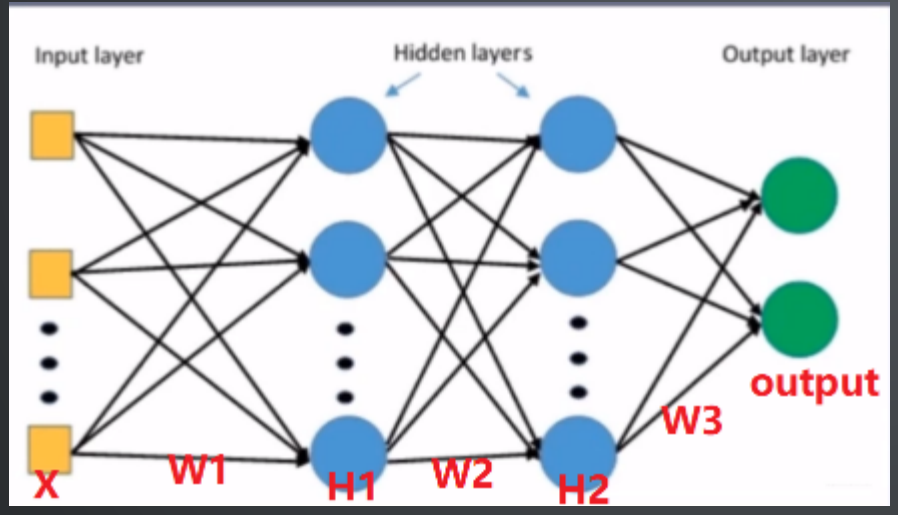
```
tensor(1., 2., 3.) torch.Size([1, 3])
tensor([[1., 1., 1.],
        [2., 2., 2.],
        [3., 3., 3.],
        [4., 4., 4.]]) torch.Size([4, 3])
tensor( 6.5000, 12.5000, 18.5000, 24.5000, g
```

这个是比较简单的线性组合

3.5-5. 激活函数层

激活函数对特征进行非线性变换， 赋予多层神经网络具有**深度**的意义。

如果没有激活函数，我们可以看一下下面的计算：



我们如果没有激活函数， 那么:

$$
\begin{aligned}
\boldsymbol{H_1} &= \boldsymbol{X} * \boldsymbol{W_1} \\
\boldsymbol{H_2} &= \boldsymbol{H_1} * \boldsymbol{W_2} \\
\mathrm{Out}\boldsymbol{put} &= \boldsymbol{H_2} * \boldsymbol{W_3} \\
&= \boldsymbol{H_1} * \boldsymbol{W_2} * \boldsymbol{W_3} \\
&= \boldsymbol{X} * (\boldsymbol{W_1} * \boldsymbol{W_2} * \boldsymbol{W_3}) \\
&= \boldsymbol{X} * \boldsymbol{W}
\end{aligned}
$$

- 这里就可以看到，一个三层的全连接层，其实和一个线性层一样。
- 由于线性运算的矩阵乘法的结合性，无论多少个线性层的叠加，其实就是矩阵的一个连乘，最后还是一个矩阵。
- 所以如果没有激活函数，再深的网络也没有意义。

下面介绍几个常用的非线性激活函数：

1. sigmoid函数

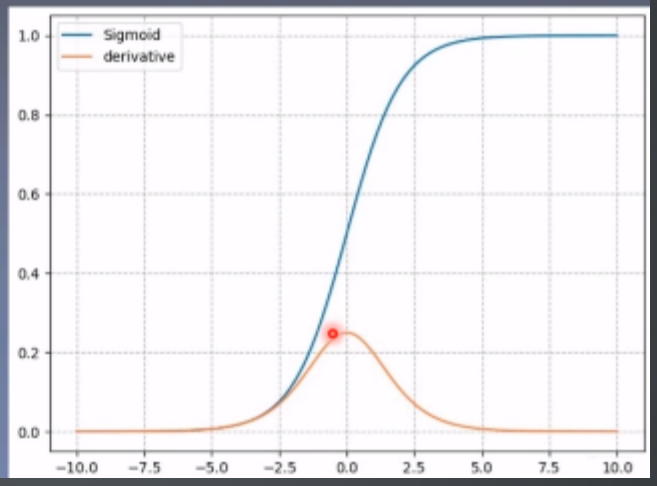
nn.Sigmoid

计算公式： $y = \frac{1}{1+e^{-x}}$

梯度公式： $y' = y * (1 - y)$

特性：

- 输出值在(0,1)，符合概率
- 导数范围是[0, 0.25],易导致梯度消失
- 输出为非0均值，破坏数据分布



- Sigmoid 函数的输出范围是 (0,1)，因此其均值不为零
- 例如，当输入服从标准正态分布时，Sigmoid 的输出均值约为 0.5。

2. nn.tanh

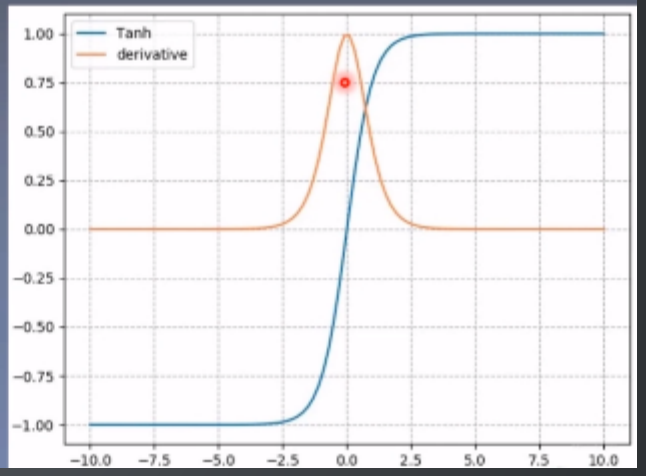
nn.tanh

计算公式：$y = \frac{\sin x}{\cos x} = \frac{e^x - e^{-x}}{e^{-} + e^{-x}} = \frac{2}{1+e^{-2x}} + 1$

梯度公式：$y' = 1 - y^2$

特性：

- 输出值在(-1,1)，数据符合0均值
- 导数范围是(0，1),易导致梯度消失



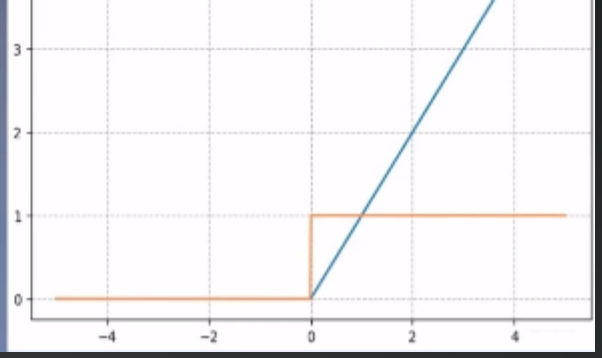
- **梯度消失问题：**
  - 当输入 x 的绝对值较大时，Tanh 的输出会趋近于 −1 或 1，此时导数接近于0。例如：
    - 若 x=3，则 f(3)≈0.995，导数为：
      f′(3)=1−(0.995)^2=0.009975
      梯度非常小。
    - 类似地，当输入为较大的负值（如 x=−3）时，导数也会接近零。
  - 在深层神经网络中，这种现象会导致梯度逐层衰减，使得靠近输入层的权重几乎无法更新，从而影响模型训练。

3. nn.ReLU

梯度公式：$y' = \begin{cases} 1, & x > 0 \\ undefined, & x = 0 \\ 0, & x < 0 \end{cases}$

特性：

- 输出值均为正数，负半轴导致死神经元
- 导数是1,缓解梯度消失，但易引发梯度爆炸



ReLU相对于前面的两个，效果要好一些， 因为不容易造成梯度消失，但是依然存在问题

死神经元问题：

- 如果输入值始终位于负半轴（x≤0），ReLU 的输出将始终为零。
- 对应的神经元在反向传播时，其梯度也会变为零（因为 ReLU 在负半轴的导数为零），导致这些神经元无法更新权重，从而“死亡”。

易引发梯度爆炸

- 因为在反向传播过程中，如果网络层数较深且权重初始化较大，ReLU 的正半轴导数恒为 1 会导致梯度不断累积放大，从而引发梯度爆炸。

所以下面就是对ReLU进行的改进。



提问：以下关于激活函数的说法，哪个是错误的?

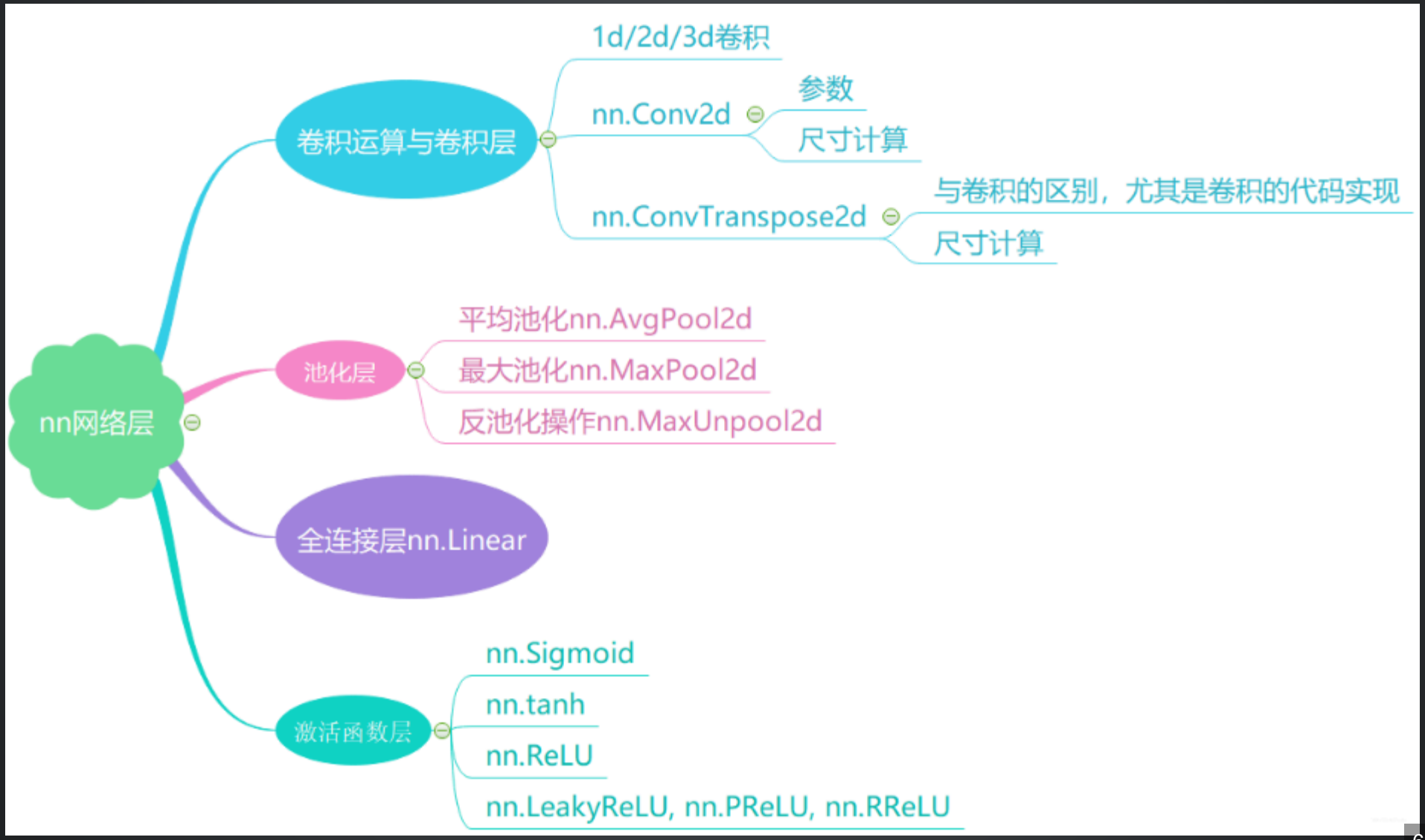
A. ReLU函数会导致死神经元问题
B. Sigmoid函数容易导致梯度消失问题
C. Tanh函数输出范围为(-1, 1)
D. 没有激活函数时，神经网络仍然可以表现出深度学习的优势

正确答案：D *解答分析：*

- ReLU函数在负半轴输出恒为0，可能导致死神经元问题（正确）。
- Sigmoid函数在输入值较大或较小时，其导数趋近于0，容易导致梯度消失问题（正确）。
- Tanh函数的输出范围确实为(-1, 1)（正确）。
- 没有激活函数时，多层神经网络等效于线性变换，无法表现出深度学习的优势（错误）.

3.5-6. 总结

下面依然是一张导图把知识拎起来：



简单的小结一下：

- 首先我们上次知道了构建神经网络的两个步骤：搭建子模块和拼接子模块。
- 而这次就是学习各个子模块的使用。

1. 从比较重要的卷积层开始， 学习了1维/2维/3维卷积到底在干什么事情，
    - 采用了动图的方式进行演示， 卷积运算其实就是通过不同的卷积核去提取不同的特征。
    - 然后学习了Pytorch的二维卷积运算及转置卷积运算，并进行了对比和分析了代码上如何实现卷积操作。

2. 是池化运算和池化层的学习

- 关于池化，一般和卷积一块使用，目的是收集和合并卷积提取的特征，去除一些冗余，
- 分为最大池化和平均池化。

3. 学习了全连接层，这个比较简单
4. 最后是非线性激活函数，比较常用的sigmoid， tanh, relu等。

模型模块基本上到这里也差不多了 - 回顾深度学习的大框架

数据模块 -> 模型模块 -> 损失函数 -> 优化器 -> 迭代训练

- 下一次开始学习:
  - 损失函数模块
  - 并且看一下常用的权重初始化方法